NMC-OPT: Intelligent Compiler Optimization for Near-Memory Computing

Malisetty Sai Nikhit, Dhayapule Kartikesh Rao, Metta Venkata Surya Sai Srikar, Aahaan Kothari, Govind Bhatia

Under the Guidance of

Prof. Bhuvaneswari M

ABSTRACT

Near-Memory Computing (NMC) and Processing-in-Memory (PIM) aim to reduce data movement by placing computation close to memory. These approaches are particularly relevant for memory-intensive applications where data transfer can dominate execution time and energy consumption.

However, determining whether a workload should execute on a host CPU or a near-memory processing unit remains challenging. Transfer overhead, memory-access patterns, workload size, and available processing resources can significantly influence the final performance.

Existing methodologies include manually optimized libraries, instruction-level offloading, and search-based tensor compilation. Although these approaches improve programmability and performance for selected workloads, they may require hardware-specific tuning or extensive candidate exploration.

The limitations of existing methods motivate a unified compiler-assisted approach that considers both computation and data movement. NMC-OPT analyzes kernel characteristics, estimates execution costs, and evaluates alternative execution strategies before recommending CPU or near-memory execution.

The proposed framework combines workload characterization, intermediate representation, analytical cost modelling, loop-transformation candidates, and bank-aware data-layout reasoning. The prototype uses a React/TypeScript/Vite frontend and a Python/FastAPI backend, with benchmark kernels including Vector Add, Reduction, ReLU, GEMV, and Fused AXPY.

The supplied draft reports CPU-only latency of 0.89 ms and direct-NMC latency of 0.77 ms, corresponding to a derived speedup of approximately 1.16×. It also reports energy values of 4800, 2000, and 1600 mJ for the CPU baseline, direct NMC, and optimized NMC, respectively. These values require validation against reproducible measurements or a documented analytical model.

Future work will integrate validated simulator or hardware measurements, improve cost-model calibration, and extend the compiler to support more complex loop structures and heterogeneous execution strategies.

KEYWORDS

Analytical Cost Modelling, Compiler Optimization, Memory-Bound Kernels, Near-Memory Computing, Processing-in-Memory, Workload Characterization

INTRODUCTION

Modern computing applications increasingly process large arrays, tensors, and data structures. In such workloads, execution time and energy consumption can be dominated by moving operands between processing units and memory rather than by arithmetic operations alone. Near-Memory Computing (NMC) and Processing-in-Memory (PIM) address this limitation by placing processing capabilities close to memory banks or within memory devices. Recent research has explored different PIM architectures, programming models, and workload-mapping techniques to reduce data movement and improve system efficiency [1]–[4].

The main problem addressed in this work is the difficulty of determining when a computational kernel should execute on a host CPU and when it may benefit from near-memory execution. Small workloads can perform poorly on an accelerator because launch and transfer overheads dominate, whereas large streaming workloads may benefit from higher effective memory bandwidth. Therefore, an effective compiler-assisted framework must consider end-to-end execution cost rather than device-side computation time alone.

Existing research has explored multiple approaches to memory-centric computation. Architecture-focused studies investigate DRAM-based processing, memory-side parallelism, and the integration of processing capabilities into high-bandwidth memory systems [1], [5]–[8]. Programming frameworks such as SimplePIM provide higher-level abstractions for distributing data and computation across processing-in-memory resources, reducing the burden of writing low-level device-specific code [9]. Other studies examine compiler support and workload mapping to make PIM systems easier to program and use [10], [11].

A second research direction focuses on automatic compiler optimization for tensor and loop-based workloads. Frameworks such as TVM and Ansor demonstrate how intermediate representations, scheduling transformations, and automated search can improve generated programs across different hardware targets [12], [13]. TensorIR provides a structured representation for expressing tensor computations and transformations, while related compiler systems explore optimized tensor scheduling and dynamic workloads [14], [15]. These approaches establish important principles for representing computations, generating optimization candidates, and evaluating alternative schedules.

Despite these advances, several limitations remain relevant to practical NMC offloading. Manually optimized libraries can deliver good performance for known workloads but may require significant effort to support new kernels. Instruction-level offloading approaches may be tied to particular hardware primitives or execution models. Search-based optimizers can incur substantial exploration costs and may not provide a straightforward explanation of why a workload should execute on a CPU or a near-memory processing unit [9]–[15].

Another limitation is the difficulty of comparing results obtained using different hardware platforms, simulation assumptions, and measurement procedures. Peak memory bandwidth alone does not determine application performance because transfer overhead, memory-access regularity, bank conflicts, synchronization, and workload size can affect end-to-end execution time. Consequently, an effective compiler-assisted approach should expose its cost assumptions and distinguish analytical estimates from measured performance.

This paper proposes NMC-OPT, a compiler-assisted framework for cost-aware near-memory offloading. The framework analyzes source kernels, extracts loop and memory-access characteristics, constructs an intermediate representation, estimates CPU and NMC execution costs, and evaluates candidate transformations. Its modular design separates source validation, workload analysis, cost modelling, candidate evaluation, and reporting. The approach is suitable for a compiler-design project because it emphasizes program representation, optimization decisions, and cost-based evaluation rather than relying exclusively on hardware-specific manual tuning.

The remainder of this paper is organized as follows. Section 2 reviews 15 research works related to PIM architectures, programming frameworks, and compiler optimization. Section 3 presents the proposed system architecture, methodology, modules, mathematical formulation, and algorithms. Section 4 describes the implementation environment and analyzes the available performance values. Section 5 concludes the paper and discusses future directions. The references and required author declarations are presented at the end.

RELATED WORKS

2.1 Mutlu et al. — A Modern Primer on Processing in Memory [1]

Proposed solution: The authors present a broad survey of processing-near-memory and processing-using-memory architectures. The paper explains how moving computation closer to memory can reduce data movement and improve performance and energy efficiency.

Methodology and tools: The work synthesizes research across memory technologies, processing architectures, applications, programming models, and system integration. It compares different PIM approaches and discusses their implications for programmability, compiler support, data placement, and system design.

Results and effectiveness: The survey establishes a cross-layer perspective on PIM and explains why data movement is an important performance bottleneck. It provides a useful foundation for identifying workloads that may benefit from near-memory execution.

Research gap: The work is a survey and does not implement an automatic kernel-level cost model for CPU-versus-NMC decisions.

2.2 Cho et al. — Main-Memory Accelerators for Bandwidth-Bound Inference [2]

Proposed solution: This work investigates the acceleration of bandwidth-bound inference workloads using main-memory processing resources. It examines how computation can be moved closer to stored data to reduce memory-transfer overheads.

Methodology and tools: The approach evaluates inference workloads against a memory-centric execution model. It considers workload characteristics, available parallelism, and the relationship between data movement and computational demand.

The evaluation examines how memory-side processing resources can support data-intensive inference. Workload properties are used to understand when the proposed execution approach may offer advantages over conventional processing.

Results and effectiveness: The study motivates classifying workloads according to memory demand and computational intensity. Its findings support the idea that execution placement should depend on workload characteristics rather than on peak compute capability alone.

Research gap: A general-purpose compiler workflow for cost-based offloading of arbitrary C loop kernels remains outside its primary scope.

2.3 Oliveira et al. — MIMDram [3]

Proposed solution: MIMDram explores an end-to-end processing-using-DRAM system designed to support high-throughput multiple-instruction, multiple-data computation. It investigates how computation can be organized around the capabilities of DRAM-based processing resources.

Methodology and tools: The work considers system organization and execution support for computation associated with DRAM. Its methodology evaluates the interaction between data-parallel workloads, available processing resources, and memory-system behavior.

The evaluation focuses on how computation is distributed and how the system architecture affects workload execution. The approach highlights the need to consider both memory organization and processing parallelism.

Results and effectiveness: The work illustrates the importance of evaluating data placement, parallel execution, and memory-system effects together. It contributes to understanding how memory-centric systems can support high-throughput workloads.

Research gap: It does not provide the same lightweight source-level workflow for comparing CPU and NMC execution costs and generating recommendations.

2.4 Devaux — The True Processing-in-Memory Accelerator [4]

Proposed solution: This work discusses the design and positioning of a practical processing-in-memory accelerator. It examines the potential of computation close to memory to reduce data movement and improve application efficiency.

Methodology and tools: The work presents a system-oriented discussion of the accelerator concept, its execution model, and the role of memory proximity. Its technical analysis provides context for understanding the relationship between memory bandwidth, supported operations, and end-to-end application behavior.

The discussion also highlights practical system considerations that influence accelerator integration. These considerations are important because theoretical bandwidth improvements do not necessarily translate directly into application-level speedups.

Results and effectiveness: The work provides useful context for evaluating practical PIM accelerators. It reinforces the importance of considering system integration and execution overheads when assessing potential benefits.

Research gap: It does not implement the modular compiler analysis and kernel-specific cost-reporting workflow targeted by NMC-OPT.

2.5 Kim et al. — Aquabolt-XL HBM2-PIM [5]

Proposed solution: Aquabolt-XL demonstrates a high-bandwidth-memory processing-in-memory direction intended to support machine-learning and other data-intensive workloads. The approach integrates processing capabilities with high-bandwidth memory.

Methodology and tools: The system combines memory technology with processing functionality and examines how memory-side computation can be integrated into an accelerator platform. Its evaluation context focuses on supported operations, workload mapping, and the relationship between memory access and computation.

The platform illustrates the importance of adapting software to the operations supported by the target device. Its design also shows that memory-side processing must be considered as part of a larger hardware and software system.

Results and effectiveness: Aquabolt-XL demonstrates the practical development of HBM-PIM technology. It supports the argument that memory-side acceleration can be useful when workloads match the target's supported operations and data-access characteristics.

Research gap: A portable compiler-level cost model for arbitrary loop kernels is not the primary purpose of the hardware platform.

2.6 Hyun et al. — Characterizing a Commercial PIM Technology [6]

Proposed solution: This work characterizes a commercial processing-in-memory technology to clarify its practical capabilities and limitations. It investigates how observed behavior differs from assumptions based solely on idealized specifications.

Methodology and tools: The methodology focuses on hardware characterization and workload performance. The evaluation studies system behavior under different execution conditions to identify factors that influence the benefits of PIM.

By examining practical system characteristics, the work provides information that can inform the calibration of cost models. Such characterization is important when estimating effective bandwidth, execution overhead, and resource utilization.

Results and effectiveness: The study motivates the use of calibrated system parameters rather than treating peak internal bandwidth as a guaranteed application-level speedup. It highlights the importance of hardware-aware evaluation.

Research gap: Hardware characterization does not itself provide an end-user source compiler that automatically recommends CPU or NMC execution.

2.7 Gómez-Luna et al. — Machine-Learning Workloads on Memory-Centric Systems [7]

Proposed solution: This research evaluates and characterizes machine-learning workloads on memory-centric computing systems. It investigates how workload properties influence performance when computation is moved closer to memory.

Methodology and tools: The study examines workload behavior under memory-centric execution and considers the interaction between computation, data movement, and available processing resources. Its evaluation emphasizes matching workload characteristics to the capabilities of the target system.

The methodology supports the analysis of data-intensive workloads rather than relying exclusively on processor throughput. This is relevant to compiler optimization because workload characterization can guide decisions about execution placement.

Results and effectiveness: The work supports workload classification as an important prerequisite for effective PIM optimization. It provides context for selecting suitable kernels and identifying data-movement bottlenecks.

Research gap: A unified compiler workflow for non-machine-learning C kernels, including cost estimation and interactive recommendations, is not its central deliverable.

2.8 Hadidi et al. — CAIRO [8]

Proposed solution: CAIRO explores compiler-assisted instruction-level offloading for selected processing-in-memory operations. Its objective is to make PIM offloading more systematic by using compiler and workload information.

Methodology and tools: The approach uses compiler support to identify opportunities for executing selected operations on PIM resources. It considers workload behavior and memory-related characteristics to determine where offloading may be beneficial.

The method is associated with a specific PIM execution model and examines the relationship between instructions, data locality, and execution placement. This provides a useful reference point for compiler-assisted offloading.

Results and effectiveness: CAIRO demonstrates that compiler support can reduce some of the burden associated with manual offloading. It also illustrates how information about program behavior can inform execution decisions.

Research gap: Its instruction-level scope differs from the whole-kernel cost comparison and multi-stage transformation workflow targeted by NMC-OPT.

Bibliographic note: Verify the complete CAIRO reference against the publisher record before submission.

2.9 Ibrahim et al. — Balanced Data Placement for GEMV Acceleration [11]

Proposed solution: This work studies balanced data placement for general matrix-vector multiplication (GEMV) using processing-in-memory resources. It focuses on distributing computation and data to improve the utilization of memory-side processing units.

Methodology and tools: The methodology considers how matrix and vector data are distributed across available processing resources. It evaluates the effect of data placement on workload execution and the utilization of memory-side parallelism.

GEMV is relevant because its execution can be limited by memory bandwidth and data movement. The work therefore provides a useful example of how placement decisions influence performance.

Results and effectiveness: The study motivates including GEMV in NMC-OPT's benchmark suite. It also supports the need to analyze data distribution rather than evaluating computation in isolation.

Research gap: Its workload-specific focus does not replace a unified analysis pipeline that supports multiple kernel families.

2.10 Gogineni et al. — SwiftRL [16]

Proposed solution: SwiftRL explores efficient reinforcement learning on real processing-in-memory systems. It investigates how reinforcement-learning workloads can be mapped to memory-side processing resources.

Methodology and tools: The study examines the characteristics of reinforcement-learning workloads and the practical requirements of executing them on PIM resources. Its evaluation considers the interaction between workload structure, available device capabilities, and execution behavior.

The work's real-system orientation is important because it emphasizes the need to validate proposed optimizations against actual platform constraints. It also broadens the application context beyond simple streaming kernels.

Results and effectiveness: SwiftRL illustrates the potential for memory-centric execution in additional application domains. It also reinforces the importance of workload-specific mapping and practical evaluation.

Research gap: A compiler front end for arbitrary loop nests and a transparent analytical CPU/NMC crossover model remain separate objectives.

2.11 Shin et al. — IMTP [18]

Proposed solution: IMTP explores search-based code generation for in-memory tensor programs. Its goal is to identify effective implementations by exploring different program and data-distribution choices.

Methodology and tools: The methodology constructs a search space over tensor-program schedules and data-distribution strategies. Compiler representations are used to describe candidates, while evaluation ranks candidate implementations according to their expected or observed effectiveness.

The approach is relevant to loop scheduling and target-aware optimization because it treats optimization as a systematic candidate-selection problem. It also illustrates how a compiler can automate decisions that would otherwise require manual tuning.

Results and effectiveness: IMTP demonstrates the potential of search-based optimization for in-memory tensor workloads. It supports the use of structured program representations and candidate evaluation.

Research gap: Search-based exploration may incur additional cost and does not inherently provide the compact analytical CPU-versus-NMC explanation emphasized by NMC-OPT.

2.12 Chen et al. — SimplePIM [9]

Proposed solution: SimplePIM provides a software framework intended to make programming real processing-in-memory systems more productive. It offers higher-level abstractions for data partitioning, iteration, and communication.

Methodology and tools: The framework provides abstractions for distributing arrays and computations across processing units and managing communication between the host and PIM resources. Its evaluation includes applications executed on a real UPMEM system.

The authors compare the framework with lower-level or hand-optimized implementations, focusing on programming effort and application performance. This provides a practical reference for understanding the benefits of higher-level PIM software abstractions.

Results and effectiveness: The work reports reductions in code size and application-dependent performance improvements. It demonstrates that a higher-level programming interface can improve productivity while retaining useful performance.

Research gap: SimplePIM primarily focuses on programming abstraction rather than automatic cross-target, cost-based execution recommendations.

2.13 Chen et al. — TVM [12]

Proposed solution: TVM presents an end-to-end optimizing compiler for deep-learning workloads. It combines program representations, scheduling, and code generation to support optimized execution across different hardware backends.

Methodology and tools: TVM uses graph-level and operator-level representations to express computations and optimization opportunities. Its scheduling infrastructure supports transformations that influence loop organization, memory access, and generated code.

The framework evaluates optimized programs across different workloads and hardware targets. This establishes an important precedent for modular compiler infrastructure and target-aware optimization.

Results and effectiveness: TVM demonstrates that compiler-driven scheduling and code generation can improve the performance and portability of machine-learning workloads. It provides a foundation for later tensor compiler research.

Research gap: General tensor optimization does not automatically account for NMC-specific transfer overhead, bank layout, and CPU-versus-NMC execution decisions.

2.14 Zheng et al. — Ansor [13]

Proposed solution: Ansor generates high-performance tensor programs through automated schedule search. It aims to discover effective implementations without requiring programmers to manually specify every optimization decision.

Methodology and tools: Ansor constructs candidate tensor schedules and explores them using a search strategy guided by cost evaluation. The system uses structured program representations to generate alternative schedules and evaluate their potential.

Its evaluation spans tensor workloads and hardware targets, illustrating the role of automated search in discovering optimizations that may be difficult to write manually.

Results and effectiveness: Ansor demonstrates the potential of automated schedule generation to identify high-performance tensor programs. It shows how search strategies can support compiler optimization.

Research gap: Search-based tuning can incur exploration costs and does not inherently provide an explicit analytical explanation of the CPU-versus-NMC crossover.

2.15 Feng et al. — TensorIR [14]

Proposed solution: TensorIR introduces a tensor-program abstraction designed to expose transformations for automatic optimization. It provides a structured representation of loops, buffers, and computational blocks.

Methodology and tools: TensorIR represents tensor computations in a form that makes transformations explicit and composable. The representation is integrated with the TVM compiler ecosystem and supports the expression and evaluation of optimized tensor programs.

The approach emphasizes compiler representations that preserve useful structural information while exposing optimization opportunities. This is relevant to NMC-OPT's use of an intermediate representation for workload analysis and candidate evaluation.

Results and effectiveness: TensorIR provides a structured basis for tensor-program optimization and supports systematic transformations. It motivates making loop structure and memory accesses explicit before evaluating alternatives.

Research gap: TensorIR is a compiler abstraction rather than a complete NMC offloading policy with a calibrated end-to-end cost model.

PROPOSED WORK / SYSTEM

3.1 Problem Formulation

NMC-OPT addresses the problem of selecting an appropriate execution strategy for computational kernels whose performance is influenced by data movement. Conventional CPU execution may be suitable for small workloads, irregular computations, or kernels with substantial data reuse. Near-memory execution may be beneficial for larger, memory-intensive workloads when its available bandwidth and parallel processing resources outweigh transfer, launch, and synchronization overheads. The execution decision must therefore consider the workload size, number of operations, memory traffic, access regularity, and target-system parameters.

The proposed system treats offloading as a cost-based compiler decision. Given a source kernel, input size, and target configuration, the framework extracts relevant program characteristics and estimates the end-to-end costs of CPU and NMC execution. It then evaluates

supported transformation candidates, such as loop tiling and bank-aware data distribution, and recommends a strategy that minimizes the estimated cost while respecting correctness and resource constraints. Since the current prototype is a compiler-assistance tool, its recommendations must be distinguished from actual execution on physical PIM hardware.

3.2 Proposed System Architecture

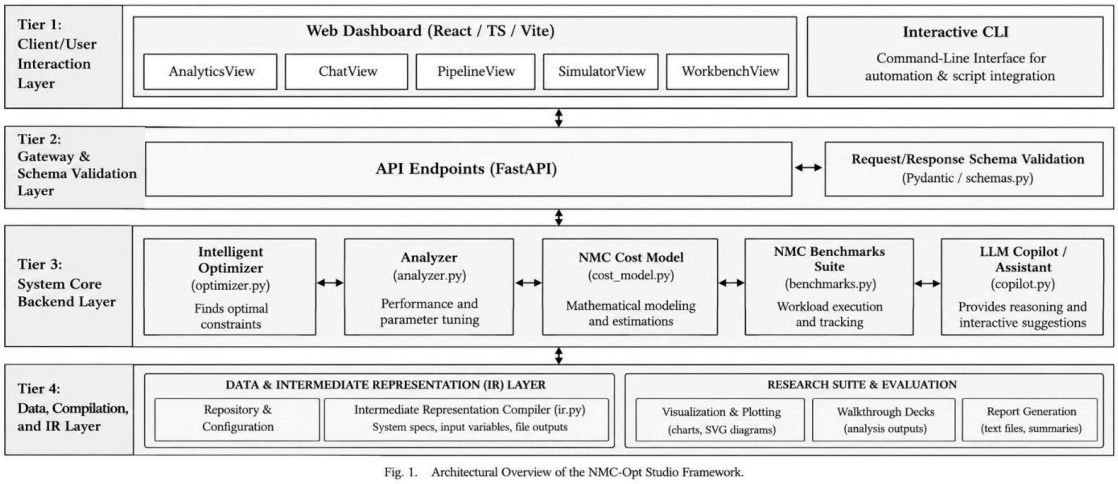


Fig. 1. Architectural Overview of the NMC-Opt Studio Framework.

Figure 1. NMC-OPT system architecture

As shown in Figure 1, the architecture begins with Tier 1: Client/User Interaction Layer, featuring the Web Dashboard (React/TS/Vite) containing views such as AnalyticsView, ChatView, PipelineView, SimulatorView, and WorkbenchView, alongside an Interactive CLI for automation and script integration. The gateway layer handles routing and security via Tier 2: Gateway & Schema Validation Layer, which encompasses FastAPI endpoints and Pydantic-based request/response schema validation (schemas.py). The core processing logic is housed in Tier 3: System Core Backend Layer, which coordinates the Intelligent Optimizer (optimizer.py) for finding optimal constraints, the Analyzer (analyzer.py) for performance and parameter tuning, the NMC Cost Model (cost_model.py) for mathematical modelling and estimations, the NMC Benchmarks Suite (benchmarks.py) for workload execution and tracking, and the LLM Copilot / Assistant (copilot.py) for reasoning and interactive suggestions. Finally, Tier 4: Data, Compilation, and IR Layer separates underlying data assets into the Data & Intermediate Representation (IR) Layer (comprising Repository & Configuration and the Intermediate Representation Compiler ir.py for system specs, input variables, and file outputs) and the Research Suite & Evaluation module for visualization, plotting, walkthrough decks, and report generation

3.3 Detailed Methodology

3.3.1 Source Processing and Kernel Analysis

The first stage accepts a supported computational kernel and validates its structure. The kernel is analyzed to identify loop bounds, array accesses, data dependencies, and computational operations. This information is used to estimate the amount of computation and data movement associated with the workload.

Compiler abstractions that represent tensor computations and their transformations provide useful context for this stage [15], [23]. For NMC-OPT, the analysis must produce enough information to determine whether the workload can be represented safely in the supported intermediate form.

3.3.2 Intermediate Representation and Polyhedral Lowering

The intermediate representation (IR) provides a structured representation of the computation that can be analyzed independently of the original source syntax. For supported affine loop nests, polyhedral methods can represent iteration domains and access relationships mathematically, enabling transformations such as loop tiling and loop reordering.

Polyhedral lowering is intended to improve locality and expose parallelism while preserving the semantics of the original computation. The transformation must respect data dependencies and valid execution order. Compiler infrastructures such as MLIR and tensor-optimization frameworks such as TVM provide related foundations for representing and transforming computational programs [15], [23], [25].

3.3.3 Analytical Cost Modelling

The cost model estimates the time required to execute a kernel on the host and near-memory processing units. It considers computational work, memory traffic, effective bandwidth, transfer latency, and synchronization overhead. The model is used to identify the conditions under which near-memory execution is expected to outperform host execution.

Let T_{CPU} denote the predicted host execution time and T_{NMC} denote the predicted total time for near-memory execution. The compiler selects the strategy with the lower estimated cost, subject to correctness and hardware constraints.

3.3.4 Execution Strategy Selection

The strategy-selection stage compares the predicted costs and chooses between host execution and near-memory execution. The decision considers input size and the relative contributions of computation, data transfer, and memory access.

The purpose of this stage is to prevent the compiler from treating every kernel as an offloading candidate. PIM research and compiler-assisted offloading studies show why workload properties and execution overhead must be considered when selecting offload candidates [7], [8], [14].

3.3.5 Memory Bank Striping and Data Placement

The memory-layout stage distributes data across the available memory banks according to the selected layout strategy. The draft proposes a 16-bank striping configuration to improve the distribution of accesses and reduce channel imbalance.

The effectiveness of bank-level striping depends on the actual memory organization, access pattern, alignment requirements, and supported transfer granularity. Data placement has also been investigated in workload-specific PIM optimization [9]. Therefore, the proposed layout must be evaluated against a clearly defined baseline before its benefits can be generalized.

3.3.6 Kernel Generation and Evaluation

After selecting the execution strategy and applying the supported transformations, the framework produces the corresponding host or near-memory execution representation. The resulting implementation is evaluated using execution time, speedup, data-movement energy estimates, and other applicable metrics.

The framework should record the input size, selected strategy, transformation configuration, execution environment, and observed measurements for each experiment. Such records are necessary to reproduce the results and determine whether the analytical model correctly predicts the preferred strategy.

3.4 Modules and Their Functions

Module	Main function	Expected output
Input validation	Checks the submitted kernel and configuration	Validated input
Kernel analysis	Extracts loop, memory-access, and workload characteristics	Kernel profile
IR construction	Creates an intermediate representation	Analyzable computation
Cost modelling	Estimates host and near-memory execution costs	Cost estimates
Strategy selection	Chooses the lowest-cost valid execution strategy	Execution decision
Polyhedral optimization	Applies supported loop transformations	Optimized loop representation
Memory-layout optimization	Distributes data across memory banks	Transformed data layout
Code generation	Produces the selected execution representation	Host or near-memory code
Evaluation and visualization	Collects and displays performance information	Metrics, graphs, and reports

3.5 Workflow and Flow Diagram

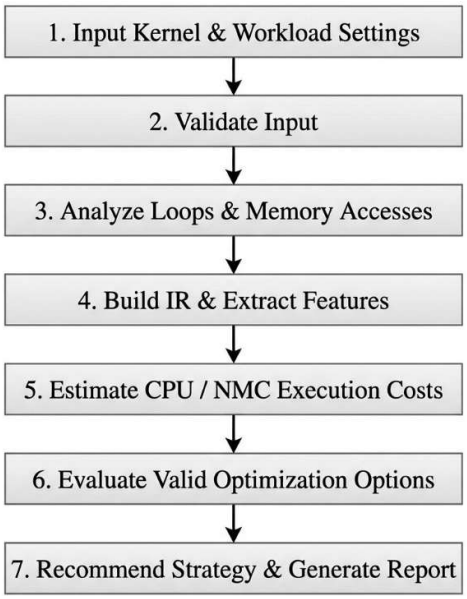


Figure 2. NMC-OPT execution workflow.

The workflow begins with a kernel and its input configuration. After validation, the framework analyzes the computation and constructs the intermediate representation. The cost model then estimates the execution costs for the available strategies. If near-memory execution is not predicted to be beneficial, the compiler selects the host path. Otherwise, it applies the supported transformations and generates the near-memory execution representation.

Both paths lead to metric collection and reporting. Comparing predicted costs with measured execution times allows the framework to evaluate the quality of its strategy-selection mechanism. This workflow is consistent with the broader objective of integrating compiler analysis and performance-driven execution selection [8], [14], [17].

3.6 Mathematical Formulation

The following equations define a conceptual cost model for NMC-OPT. The parameters must be calibrated or measured for the actual target platform.

Equation (1): Host execution time

$$T_{\text{CPU}} = T_{\text{compute,CPU}} + T_{\text{memory,CPU}} + T_{\text{overhead,CPU}}$$

Here, T_compute,CPU is the host computation time, T_memory,CPU represents memory-related delay, and T_overhead,CPU includes other relevant host-side overheads.

Equation (2): Near-memory execution time

$$T_{NMC} = T_{launc} + T_{transfer} + T_{compute,NMC} + T_{sync}$$

The total near-memory time includes kernel-launch overhead, data-transfer time, near-memory computation time, and synchronization overhead. The terms must be defined consistently to avoid counting any overhead twice.

Equation (3): Data-transfer time

$$T_{transfer} = \frac{D_{in}}{B_{in}} + \frac{D_{out}}{B_{out}}$$

Here, D_in and D_out are the input and output data volumes, while B_in and B_out are the corresponding effective transfer bandwidths.

Equation (4): Offloading decision

$$S^* = arg\ min_{S \in \{CPU,NMC\}} \hat{T}(S)$$

Here, S denotes an execution strategy and $\hat{T}(S)$ denotes its predicted total execution time. The selected strategy minimizes predicted cost among the supported alternatives.

Equation (5): Speedup

$$Speedup = \frac{T_{baseline}}{T_{optimized}}$$

A speedup greater than 1 indicates that the optimized implementation is faster than the specified baseline. The baseline and optimized measurement conditions must be consistent.

Equation (6): Arithmetic intensity

$$I = \frac{F}{D}$$

Here, F is the number of floating-point operations and D is the number of bytes transferred. Arithmetic intensity helps characterize whether a kernel is likely to be limited by computation throughput or memory bandwidth.

Equation (7): Effective bandwidth

$$B_{eff} = \frac{D}{T_{memory}}$$

Here, D represents the measured or estimated data volume and T_memory represents the corresponding memory-transfer interval.

Equation (8): Relative energy reduction

$$E_{reduction} = \frac{E_{baseline} - E_{optimized}}{E_{baseline}} \times 100\%$$

Here, E_baseline and E_optimized represent energy consumption for the baseline and optimized configurations, respectively. The energy boundary and estimation method must remain consistent across the comparison.

These equations establish a basis for comparing execution strategies, characterizing memory behavior, and reporting performance. The cost-model terms must be calibrated using the actual platform or a clearly described simulation model; they should not be interpreted as proof of measured performance by themselves.

3.7 Algorithms

Algorithm 1: Cost-Based Execution Strategy Selection

Input: Kernel profile, input size, estimated host cost, estimated near-memory cost.

Output: Selected execution strategy.

1. Read the kernel profile and input configuration.
2. Validate the kernel and its supported operations.
3. Estimate host execution cost.
4. Estimate near-memory execution cost, including transfer overhead.
5. If the near-memory strategy is supported and its estimated cost is lower:
   Select near-memory execution.

   Otherwise: Select host execution.
6. Return the selected strategy and cost estimates.

This algorithm formalizes the execution-placement decision. It avoids assuming that near-memory execution is always beneficial and instead selects the strategy with the lowest estimated cost. The decision follows the motivation of compiler-assisted PIM offloading [8], [14].

Algorithm 2: Memory-Aware Kernel Optimization

Input: Validated kernel, intermediate representation, memory-layout configuration.

Output: Transformed kernel representation and selected layout.

1. Construct the intermediate representation of the kernel.
2. Identify supported loop nests and memory-access patterns.
3. Determine the applicable loop transformations.
4. Generate candidate layouts for the supported memory-bank configuration.
5. Estimate the cost of each valid candidate.
6. Reject candidates that violate correctness or target constraints.
7. Select the lowest-cost valid candidate.
8. Generate the corresponding execution representation.
9. Return the optimized representation and configuration.

This algorithm describes the intended optimization sequence. In an implementation, each candidate must preserve program semantics, and cost estimates should be validated against measured results. Search-based tensor compilers provide relevant context for exploring transformation candidates [21], [22], [23].

3.8 Proposed Diagrams and Illustrations

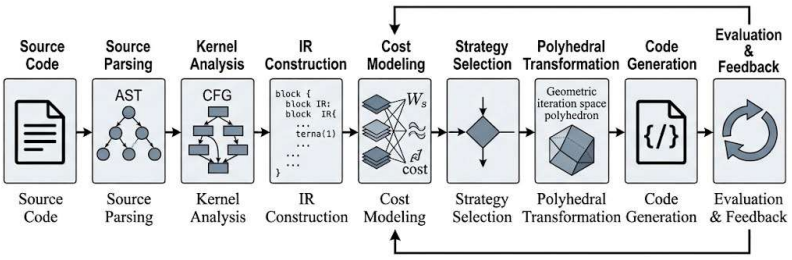


Figure 3. Compiler optimization pipeline.

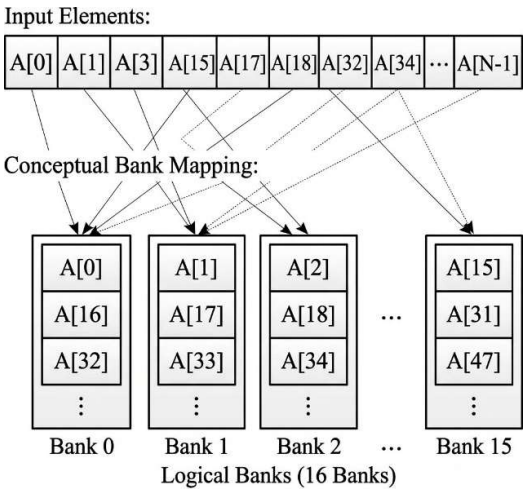


Figure 4. Memory-bank striping.

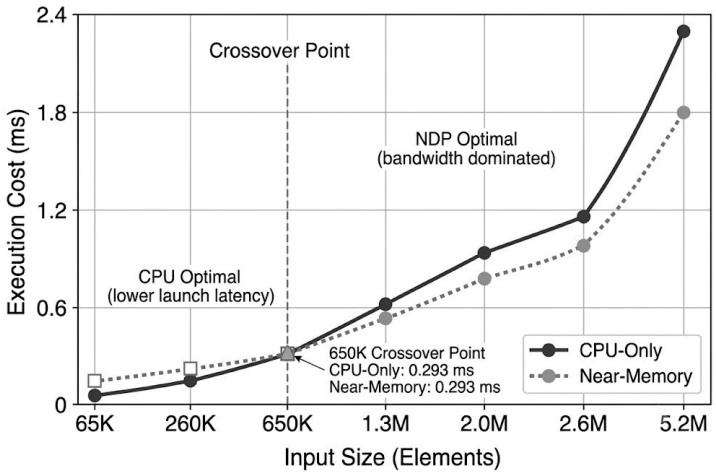


Figure 5. Cost-based offloading decision

IMPLEMENTATION AND RESULT ANALYSIS

4.1 Implementation Tools and Experimental Configuration

The NMC-OPT draft identifies MacSim, VaultSim/DRAMSim2, the Structural Simulation Toolkit (SST), Apache TVM, and a FastAPI/Vite frontend as tools associated with the implementation or evaluation workflow. These tools serve different purposes: microarchitecture and memory-system simulation, compiler-based optimization, and application-level interaction. MacSim, DRAMSim2, and SST are relevant to architectural evaluation, while TVM provides a framework for compiler-based optimization [15], [27], [28], [29].

The final implementation description must specify which of these tools were actually executed, their versions, the target architecture, the input workloads, and the commands used. A tool appearing in the design or planning stage is not sufficient evidence that it was used to generate the reported measurements. The host configuration, memory parameters, simulator settings, DPU count, input sizes, and measurement procedure should be recorded so that the experiments can be reproduced.

4.2 Implementation Screenshot

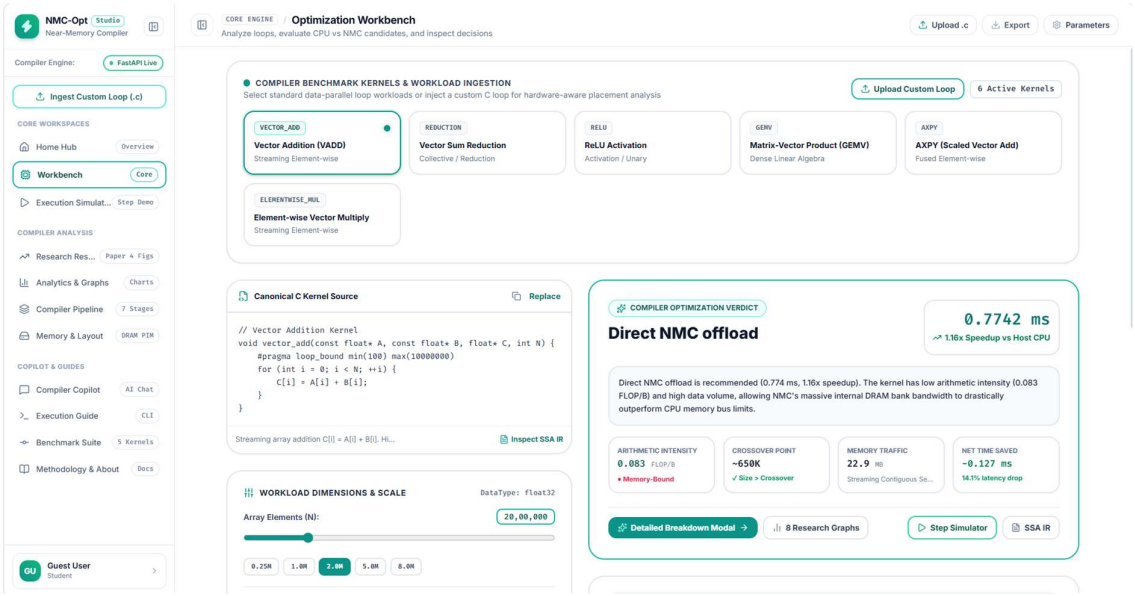


Figure 6. NMC-OPT Studio implementation interface.

4.3 Parameter Setup

Parameter	Value in draft	Purpose
Memory-bank configuration	16 banks	Evaluate bank-level data distribution
Input-size range	65K to 5.2M elements	Evaluate scaling and crossover behavior
Reported crossover	Approximately 650K elements	Compare estimated host and near-memory costs
Host-only latency	Approximately 0.89 ms	Baseline execution time in the reported example
Direct NMC latency	Approximately 0.77 ms	Near-memory execution time in the reported example
Interconnect bandwidth sweep	16–128 GB/s	Evaluate transfer-bandwidth sensitivity
Example link bandwidth	96 GB/s	Illustrate the reported sensitivity experiment
DPU scaling range	64–2048 DPUs	Evaluate parallel scaling
Reported DPU sweet spot	256–512 DPUs	Candidate range for parallel efficiency

Data-movement energy baseline	4800 mJ	Baseline energy figure in the draft
Direct NMC energy	Approximately 2000 mJ	Reported near-memory configuration
Optimized NMC energy	Approximately 1600 mJ	Reported interleaved configuration

4.4 Performance Evaluation Metrics

4.4.1 End-to-End Latency and Speedup

End-to-end latency measures the time required to complete a workload, including the overheads included in the declared measurement boundary. The draft reports a CPU-only latency of approximately 0.89 ms and a direct NMC latency of approximately 0.77 ms.

Using these reported values, the illustrative speedup is

$$\text{Speedup} = \frac{0.89}{0.77} \approx 1.16\times$$

This corresponds to a reduction in latency of approximately 13.5% relative to the reported baseline. The comparison is meaningful only if both times represent the same workload and include comparable execution overheads.

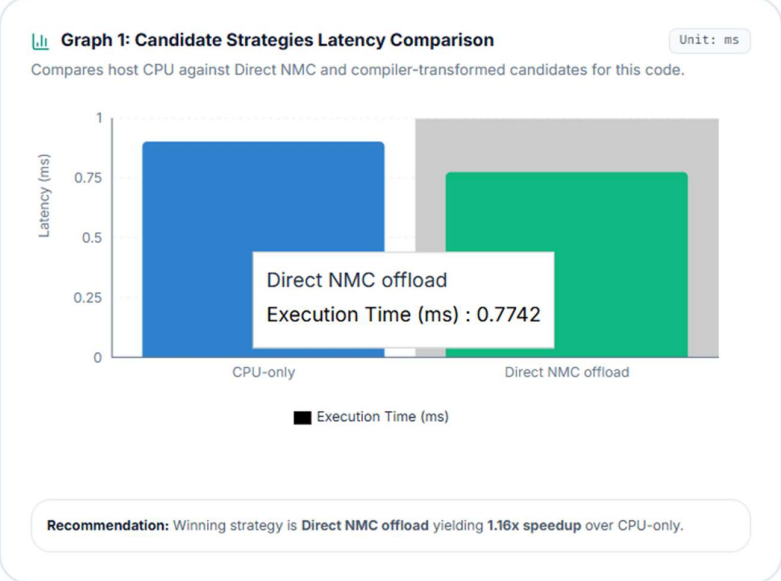


Figure 7. End-to-end execution latency comparison.

4.4.2 Input-Size Scaling and Crossover Behavior

The input-size experiment examines the relationship between workload size and the relative costs of host and near-memory execution. The draft identifies an approximate crossover point at 650K elements. Below this point, fixed launch and transfer overheads are reported to favor

CPU execution; above it, the near-memory path is reported to benefit from increased internal bandwidth.

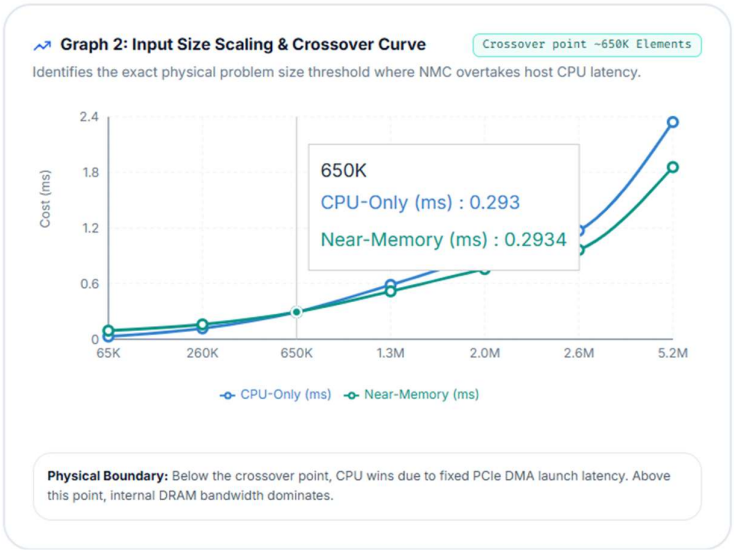


Figure 8. Execution latency as a function of input size.

4.4.3 Latency Component Breakdown

The latency-breakdown experiment separates execution time into computation, data layout, synchronization, and data transfer. The draft proposes comparing the host CPU and near-memory paths to determine which components dominate total execution time.

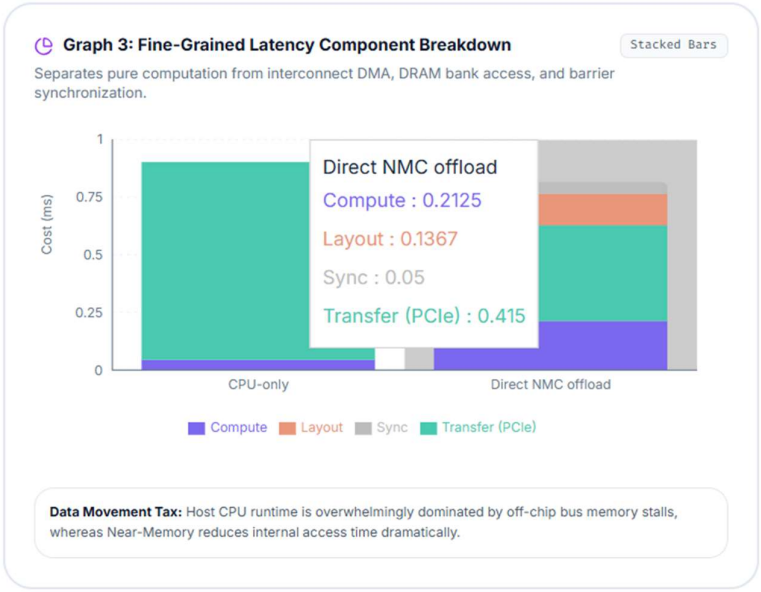


Figure 9. Breakdown of execution latency by component.

4.4.4 Data-Movement Energy Consumption

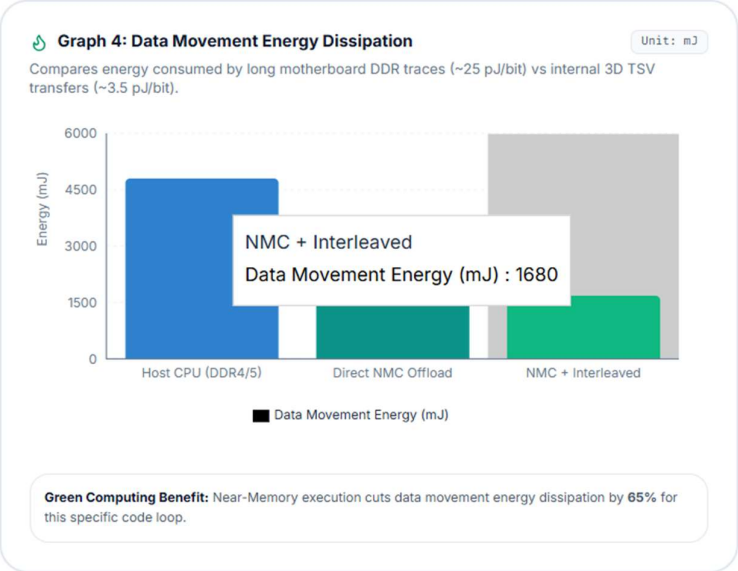


Figure 10. Data-movement energy across execution strategies.

4.4.5 Arithmetic Intensity and Roofline Analysis

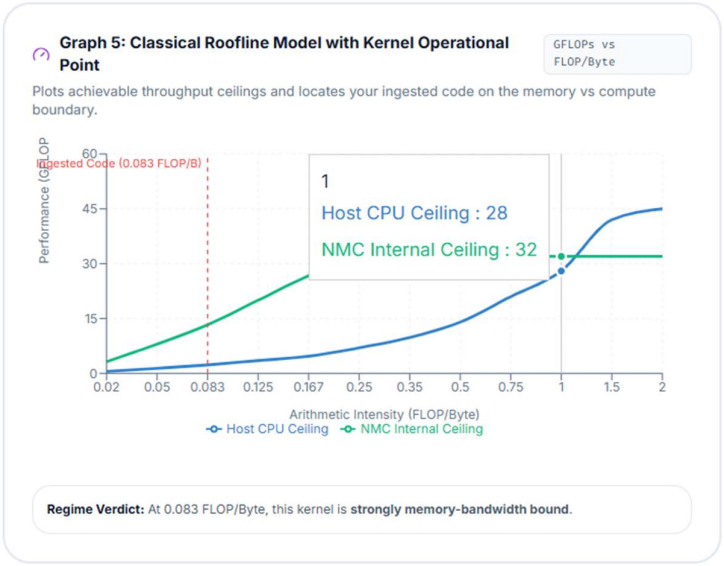


Figure 11. Roofline model and kernel operating point.

4.4.6 Interconnect Bandwidth Sensitivity

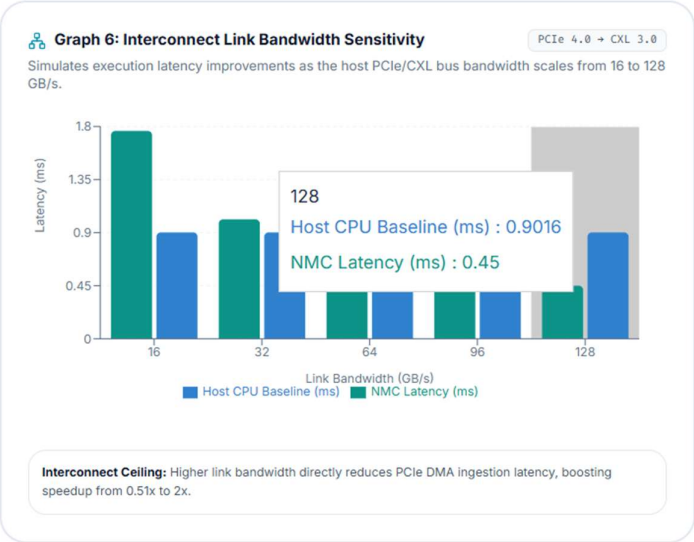


Figure 12. Interconnect bandwidth sensitivity.

4.4.7 Bank-Conflict and Layout Analysis



Figure 13. Memory-layout latency comparison.

4.4.8 DPU Scaling and Parallel Efficiency

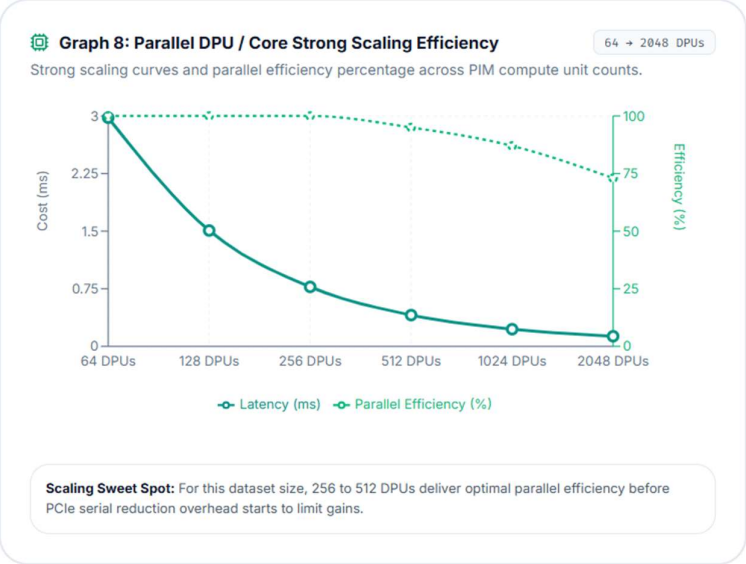


Figure 14. DPU scaling and parallel efficiency.

Parallel efficiency should be defined relative to a stated baseline. For example, if P is the number of processing units and $T(P)$ is the measured runtime, strong-scaling efficiency relative to P_0 units can be expressed as

$$\eta(P) = \frac{T(P_0)}{T(P)} \frac{P_0}{P}$$

The reference configuration P_0 must be specified. The reported sweet spot should be justified by the measured runtime and efficiency curves rather than assumed in advance.

4.5 Comparative Analysis with Related Work

Comparison dimension	CAIRO [8]	IMTP [13]	NMC-OPT (this work)
Main focus	Compiler-assisted, instruction-level PIM offloading using HMC-atomic instructions, evaluated on graph workloads	Search-based code generation for in-memory tensor programs on UPMEM	Analytical cost modelling, IR lowering, and bank-level data placement for CPU-vs-NMC decisions
Optimization approach	Profiling-based compile-time decision model: eligibility tests, LLC miss-ratio analysis, and bandwidth-savings analysis. Machine constants come from	Autotuning in TVM: evolutionary search with a learned cost model over a joint host and kernel schedule space. Adds PIM-aware passes (DMA-aware boundary-check	Cost-based strategy selection (CPU vs NMC) with loop-transformation candidates and 16-bank striping

	micro-benchmarks and linear regression.	elimination, loop-bound tightening, invariant branch hoisting).	
Execution platform	Simulated CPU+HMC 2.0 (16 out-of-order cores, 8 GB HMC with 32 vaults) and GPU+HMC (15 SMs). Uses MacSim, VaultSim, and SST.	Real UPMEM server: dual-socket Xeon Gold 5220R, 2048 DPUs (32 ranks), TVM 0.13.0, UPMEM SDK 2021.3.0	Python 3.14.3, FastAPI 0.142.2, pycparser 3.00, pytest 9.1.1, Node.js v24.14.1, React 19.2.8, Vite 8.3.4
Latency	Up to 2× speedup on PIM-beneficial workloads and no slowdown on the others. 63 workloads (27 CPU, 36 GPU).	Average 3.05× over PrIM, 1.48× over PrIM(E), 1.67× over PrIM+search, and 3.3× over SimplePIM. Up to 8.21× on GPT-J layers.	0.89 ms (CPU-only) vs 0.77 ms (direct NMC), a derived speedup of ≈ 1.16 ×.
Energy	Not reported quantitatively	Not reported	4800 mJ (CPU baseline), 2000 mJ (direct NMC), 1600 mJ (optimized NMC).
Research gap addressed	Instruction-level and fixed-function scope, profiling dependence, and simulated hardware. Does not make whole-kernel offloading decisions.	UPMEM-specific, needs hardware measurements for the search, and provides no compact analytical CPU-vs-NMC explanation	NMC-Opt Studio uses static source-level analysis and analytical cost modeling to estimate workload crossover thresholds and guide efficient NMC offloading strategy decisions before execution.

4.6 Discussion and Limitations

The reported results suggest that NMC-OPT may reduce execution cost for selected memory-intensive workloads when the transfer and launch overheads are outweighed by the benefits of near-memory processing. The input-size and interconnect sensitivity experiments are particularly relevant because they examine when the preferred execution strategy changes.

However, the strength of these conclusions depends on the validity of the cost model, the fidelity of the experimental environment, and the consistency of the baseline measurements. The final manuscript should identify which results were obtained from hardware, which were generated by simulation, and which were estimated analytically. Reproducible configuration

files, workload definitions, raw results, and calculation procedures should be provided wherever possible.

CONCLUSION AND SCOPE FOR FUTURE WORK

This paper presents NMC-OPT, a compiler-assisted framework intended to automate execution-cost estimation, optimization, and offloading decisions for near-memory computing workloads. The framework combines analytical cost modelling, intermediate representation analysis, polyhedral transformations, and bank-level data placement.

Existing PIM frameworks and compiler optimization systems address important aspects of near-memory execution, including instruction-level offloading, tensor scheduling, and workload-specific acceleration [8], [13], [14], [22]. However, execution overheads, memory layout, and workload characteristics continue to influence whether near-memory execution is beneficial.

The proposed approach aims to integrate these concerns into a unified optimization workflow. Its principal advantage is the ability to compare predicted execution costs before selecting a host or near-memory execution strategy. The framework also provides a basis for evaluating loop transformations and data-placement choices.

The draft identifies MacSim, DRAMSim2, SST, Apache TVM, and a FastAPI/Vite application interface as tools associated with the implementation and evaluation workflow [15], [27], [28], [29]. The final manuscript should identify the tools actually used in each experiment and provide the corresponding configuration details.

The supplied prototype results report speedups of 1.16× to 1.66×, reduced data-movement energy, improvements associated with bank-interleaved placement, and a candidate parallel-efficiency sweet spot of 256–512 DPUs. These results should be treated as preliminary until verified against the final experimental dataset. The evaluation should also clarify the measurement boundaries, workload configurations, and distinction between measured and simulated values.

Future work will investigate the integration of graph-level intermediate representations, including TVM Relax, to support more complex computational graphs and automated workload partitioning. Additional research may examine dynamic workload sizes, more general memory-access patterns, and cost-model calibration across heterogeneous platforms. Extending the framework to emerging CXL-based memory systems and other near-memory technologies is another possible direction. Finally, systematic validation against existing PIM frameworks and reproducible benchmarks will be essential for establishing the generality and effectiveness of NMC-OPT.

REFERENCES

[1] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A modern primer on processing in memory," in *Emerging Computing: Looking Beyond Moore and Von Neumann*, Berlin, Germany: Springer, 2022, pp. 171–243.

[2] B. Y. Cho, J. Jung, and M. Erez, "Accelerating bandwidth-bound deep learning inference with main-memory accelerators," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2021, pp. 1–15.

[3] G. F. Oliveira et al., "MIMDram: An end-to-end processing-using-DRAM system for high-throughput MIMD computing," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Mar. 2024, pp. 186–203.

[4] F. Devaux, "The true processing in memory accelerator," in *IEEE Hot Chips 31 Symp. (HCS)*, Aug. 2019, pp. 1–24.

[5] J. H. Kim et al., "Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond," in *IEEE Hot Chips 33 Symp. (HCS)*, Aug. 2021, pp. 1–26.

[6] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future PIM architectures by demystifying a commercial PIM technology," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Mar. 2024, pp. 263–279.

[7] J. Gómez-Luna et al., "Evaluating machine learning workloads on memory-centric computing systems," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, Apr. 2023, pp. 35–49.

[8] R. Hadidi, L. Nai, H. Kim, and H. Kim, "CAIRO: A compiler-assisted technique for enabling instruction-level offloading of processing-in-memory," *ACM Trans. Architecture Code Optim.*, vol. 14, no. 4, Art. no. 48, pp. 48:1–48:25, Dec. 2017.

[9] M. A. Ibrahim, M. Islam, and S. Aga, "Balanced data placement for GEMV acceleration with processing-in-memory," in *Proc. Int. Symp. Memory Syst. (MEMSYS)*, Oct. 2024, pp. 112–125.

[10] K. Gogineni et al., "SwiftRL: Towards efficient reinforcement learning on real processing-in-memory systems," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, May 2024, pp. 217–229.

[11] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware PIM architecture," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Architecture (ISCA)*, Jun. 2015, pp. 336–348.

[12] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, "GraphPIM: Enabling instruction-level PIM offloading in graph computing frameworks," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architecture (HPCA)*, Feb. 2017, pp. 457–468.

[13] Y. Shin, D. Kang, and H. Sung, "IMTP: Search-based code generation for in-memory tensor programs," arXiv preprint arXiv:2412.19630, Dec. 2024.

[14] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A software framework for productive and efficient processing-in-memory," in *Proc. Int. Conf. Parallel Architectures Compiler Tech. (PACT)*, Oct. 2023, pp. 99–111.

[15] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Oct. 2018, pp. 578–594.

[16] J. Gómez-Luna et al., "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52565–52608, May 2022.

[17] A. A. Khan et al., "CINM: A compilation infrastructure for heterogeneous compute in-memory and compute near-memory," arXiv preprint arXiv:2311.02103, Nov. 2023.

[18] C. Giannoula et al., "SparseP: Efficient sparse matrix vector multiplication on real processing-in-memory architectures," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, Jul. 2022, pp. 288–291.

[19] S. Lee et al., "A 1nm 1.25V 8Gb 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.

[20] C. Lim et al., "Design and analysis of a processing-in-DIMM join algorithm: A case study with UPMEM DIMMs," *Proc. ACM Manag. Data*, vol. 1, no. 2, Art. no. 147, pp. 1–26, Jun. 2023.

[21] B. Zheng et al., "DietCode: Automatic optimization for dynamic tensor program," in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, vol. 4, Aug. 2022, pp. 208–222.

[22] L. Zheng et al., "Ansor: Generating high-performance tensor programs for deep learning," in *Proc. 14th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Oct. 2020, pp. 863–879.

[23] S. Feng et al., "TensorIR: An abstraction for automatic tensorized program optimization," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Oper. Syst. (ASPLOS)*, Mar. 2023, pp. 804–817.

[24] A. Boroumand et al., "LazyPIM: An efficient cache coherence mechanism for processing-in-memory," *IEEE Comput. Architecture Lett.*, vol. 16, no. 1, pp. 14–17, Jan.–Jun. 2017.

[25] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2021, pp. 2–14.

[26] J. Lin et al., "SongC: A compiler for hybrid near-memory and in-memory many-core architecture," *IEEE Trans. Comput.*, vol. 73, no. 10, pp. 2420–2433, Oct. 2024.

[27] H. Kim, R. Hadidi, and L. Nai, "MacSim: A cycle-level microarchitecture simulator for heterogeneous systems," Georgia Institute of Technology, Atlanta, GA, USA, Tech. Rep., 2018.

[28] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "DRAMSim2: A cycle accurate memory system simulator," *IEEE Comput. Architecture Lett.*, vol. 10, no. 1, pp. 16–19, Jan.–Jun. 2011.

[29] A. Rodrigues et al., "The Structural Simulation Toolkit," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 38, no. 4, pp. 37–42, Mar. 2011.

[30] Y. Shin, J. Park, S. Cho, and H. Sung, "PIMFlow: Compiler and runtime support for CNN models on processing-in-memory DRAM," in *Proc. ACM/IEEE Int. Symp. Code Gener. Optim. (CGO)*, Feb. 2023, pp. 249–262.

AUTHOR CONTRIBUTIONS

Malisetty Sai Nikhit (24BCE2642): Software (Full-Stack System Architecture, Frontend and Backend Development), Data Curation (Visual Analytics and Interactive Data Presentation), System Optimization, and Writing (Manuscript Review and Editing).

Dhayapule Kartikesh Rao (24BCE0241): Software (UI/UX Design and Implementation, Frontend Workbench Development), Validation (Software Testing and Verification), Analytics and Graph Implementation, and Writing (Manuscript Drafting and Preparation).

Metta Venkata Surya Sai Srikar (24BCE0310): Conceptualization and Methodology (System Architecture, Cost Modelling, Memory Layout), Software (Website Implementation and Backend API Integration), Validation (Software Testing and Verification), and Formal Analysis (Result and Parameter Analysis).

Aahaan Kothari (24BDS0273): Mathematical Formulation, Cost Modelling, Memory-Traffic Analysis, Optimization Evaluation, Data-Movement Cost Analysis, Interconnect and Energy-Dissipation Validation, Research Paper Analysis.

Govind Bhatia (24BCE0648): Formal Analysis, Software (SSA-Based Intermediate Representation, Compiler Transformations, NMC Loop Fusion Logic, NMC Tiling Logic), Investigation, Strategy Evaluation, Research Paper Analysis.

All authors reviewed, revised, and approved the final manuscript.

ACKNOWLEDGEMENTS

We sincerely thank **Prof. Bhuvaneswari M**, our Compiler Design faculty at Vellore Institute of Technology, Vellore, for her valuable guidance, constructive feedback, and encouragement throughout the development of this project. Ma'am's suggestions and support contributed to improving the design, implementation, and presentation of the proposed work.

FUNDING

This research received no external funding from any public, commercial, or non-profit funding agency.

COMPETING INTERESTS

The authors declare that they have no competing financial or personal conflicts of interest.

DATA AND CODE AVAILABILITY

The complete source code, compiler passes, analytical cost engines, benchmarks, and reproduction scripts are openly available at: https://github.com/MSN-1307/NMC-OPT-CD.